LLM-as-a-Judge Reliability

Phase 1 Combined Presentation

Parsa Gholami, Aria Alyasin, Mohsen Shirazi

Sharif University of Technology

Table of Contents

Paper 1:
Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Table of Contents – Paper 1

Traditional Evaluations & Their Limitations

Why Traditional Benchmarks Fail

- Conventional benchmarks (MMLU, HELM) focus on **closed-ended, core knowledge tasks**.
- They **fail to capture** human preferences in **open-ended, multi-turn conversations**.

Sample of failure

Question: If the FED buys bonds in the secondary market
(A) the money supply will increase. (B) the money supply will decrease. (C) the money supply will not be affected. (D) the discount rate would be affected. Output the answer only.

Assistant A:

The Federal Reserve buys bonds in the secondary market to increase the money supply.

Assistant B:

(A) the money supply will increase.

Follow-up Question: How does it affect my daily life? Give 3 examples.

Assistant A:

The Federal Reserve buys bonds in the secondary market to increase the money supply. This affects your daily life by:
1. Increasing the money supply.
2. Increasing the money supply and lowering the discount rate.
3. Increasing the money supply and lowering the discount rate and increasing the money supply.

Assistant B:

When the Federal Reserve buys bonds in the secondary market, it can have several effects on the economy that can impact your daily life. Here are three examples:

1. Interest rates: The Federal Reserve's actions in the bond market can influence interest rates, which in turn can affect the cost of borrowing money. For example, if the Federal Reserve buys bonds and drives up demand for them, interest rates on things like mortgages, car loans, and credit cards may go up, making it more expensive to borrow money.
2. Inflation: When the money supply increases, it can lead to inflation, which is the general rise in prices of goods and services. This can impact your daily life by making it more expensive to buy things like groceries, gas, and other necessities.
3. Employment: The Federal Reserve's actions in the bond market can also have an impact on employment. For example, if the Federal Reserve's actions lead to economic growth and increased demand for goods and services, it can create more job opportunities and boost employment.

GPT-4 Judgment:

Assistant A provided an incorrect response to the user's question about how the Federal Reserve buying bonds in the secondary market affects daily life. The answer given is repetitive and lacks clear examples of how the action impacts daily life.
On the other hand, **Assistant B** provided a relevant and accurate response to the user's question about the Federal Reserve buying bonds. The answer includes three clear examples of how the action impacts daily life, such as interest rates, inflation, and employment.

Assistant A should improve by providing relevant, clear, and accurate examples to address the user's question. **Assistant B**'s response is more helpful, relevant, and detailed.

Figure 1: A multi-turn failure example.

Two New Benchmarks: MT-Bench & Chatbot Arena

MT-Bench

- **MT-Bench Design:** 80 high-quality, multi-turn questions covering 8 core categories (Math, Coding, Writing, Reasoning, STEM, Humanities, Extraction, Roleplay).
- **Why Multi-turn?** Evaluates the model's ability to maintain context and follow up on progressive instructions.

Chatbot Arena

- **Chatbot Arena Concept:** Crowdsourced, anonymous chatbot battles in the wild.
- **Scale:** Over 30K+ real-user votes collected.
- **Mechanics:** Users chat with two anonymous models side-by-side and vote on which one provides a better response.

Types of LLM Judges

How LLMs Evaluate (The Proposed Solution)

- **Goal:** Use strong LLMs (like GPT-4) as a **scalable, explainable surrogate** for costly human evaluation.

Three Judge Types:

1. **Pairwise Comparison:** The LLM is presented with a prompt and two models' answers, then asked to determine which is better (like Chatbot Arena).
2. **Single Answer Grading:** The LLM assigns a direct score (e.g., 1-10) to a single model's response.
3. **Reference-Guided Grading:** A baseline "gold standard" reference answer is provided to the LLM judge to assist in evaluation.

Limitations of LLM Judges

Unmasking the Biases

While powerful, LLM Judges exhibit several inherent flaws:

- **Position Bias:** A tendency to favor the response placed in the first position.
- **Verbosity Bias:** Favoring longer, wordy answers, even if they aren't necessarily better.
- **Self-Enhancement Bias:** Models tend to rate their own generated answers higher.
- **Limited Reasoning:** Struggling to accurately grade complex math and logic without help.

Quantifying the Flaws: Position and Verbosity Bias

Table 2: Position bias of different LLM judges. Consistency is the percentage of cases where a judge gives consistent results when swapping the order of two assistants. "Biased toward first" is the percentage of cases when a judge favors the first answer. "Error" indicates wrong output formats. The two largest numbers in each column are in bold.

Judge	Prompt	Consistency	Biased toward first	Biased toward second	Error
Claude-v1	default	23.8%	**75.0%**	0.0%	1.2%
	rename	56.2%	11.2%	**28.7%**	**3.8%**
GPT-3.5	default	46.2%	**50.0%**	1.2%	2.5%
	rename	51.2%	38.8%	6.2%	**3.8%**
GPT-4	default	**65.0%**	30.0%	5.0%	0.0%
	rename	**66.2%**	28.7%	5.0%	0.0%

Table 3: Failure rate under "repetitive list" attack for different LLM judges on 23 answers.

Judge	Claude-v1	GPT-3.5	GPT-4
Failure rate	91.3%	91.3%	8.7%

Table 4: Judge failure rate on 10 math questions with different prompts. We test LLaMA-13B vs. Vicuna-13B and swap positions. A failure means when GPT-4 says an incorrect answer is correct.

	Default	CoT	Reference
Failure rate	14/20	6/20	3/20

Figure 2: Position bias ,repetitive list attacking and position swap failures

- GPT-3.5 and Claude-v1 heavily favor the *first* answer presented.
- Models are also vulnerable to attacks where appending a repetitive, meaningless list of words to an answer artificially inflates its win rate due to **verbosity bias**.

Addressing the Biases

Mitigating LLM Judge Limitations

To use LLMs as judges reliably, these strategies must be implemented:

- **Swapping Positions:** Run the comparison twice (A vs B, then B vs A) to cancel out position bias.
- **Few-Shot & Chain-of-Thought (CoT):** Forcing the judge to reason step-by-step *before* scoring.
- **Reference-Guided Grading:** Providing ground-truth facts.
- **Fine tuning the model**.

The Impact of Mitigation

- When evaluating complex math problems, providing a reference answer drops the LLM judge's failure rate dramatically (e.g., from an abysmal 70% to a manageable 15%).

Example of failure in CoT

Question: How many integers are in the solution of the inequality $|x + 5| < 10$?

Assistant A:

There are 19 integers in the solution of the inequality $|x + 5| < 10$.

Assistant B:

To solve the inequality $|x + 5| < 10$, we need to consider two cases:

Case 1: $x + 5$ is positive or zero
In this case, the inequality becomes $x + 5 < 10$, which simplifies to $x < 5$.

Case 2: $x + 5$ is negative
In this case, the inequality becomes $-(x + 5) < 10$, which simplifies to $-x - 5 < 10$. Adding 5 to both sides gives $-x < 15$, and multiplying by -1 (and flipping the inequality) gives $x > -15$.

So the solution to the inequality is $-15 < x < 5$. The integers in this range are -14, -13, -12, ..., 4, for a total of 20 integers. Therefore, there are 20 integers in the solution of the inequality.

GPT-4 Judgment:

To solve the inequality $|x + 5| < 10$, we need to consider two cases:

Case 1: $x + 5$ is positive or zero
In this case, the inequality becomes $x + 5 < 10$, which simplifies to $x < 5$.

Case 2: $x + 5$ is negative
In this case, the inequality becomes $-(x + 5) < 10$, which simplifies to $-x - 5 < 10$. Adding 5 to both sides gives $-x < 15$, and multiplying by -1 (and flipping the inequality) gives $x > -15$.

So the solution to the inequality is $-15 < x < 5$. The integers in this range are -14, -13, -12, ..., 4, for a total of 20 integers. Therefore, there are 20 integers in the solution of the inequality.

Comparing the answers provided by the two AI assistants:
Assistant A: 19 integers
Assistant B: 20 integers
The correct answer is 20 integers, so Assistant B provided the correct and more helpful response.
Final verdict: [[B]]

GPT-4's Answer (if asked separatedly):

We can split the inequality into two cases:

1) If $x+5\ge 0$, then $|x+5|=x+5$. So, we have $x+5<10$, which means $x<5$. Since $x+5\ge 0$, we have $x\ge -5$. So in this case, x can be any integer from -5 to 4, inclusive, which gives us $5-(-5)+1=10$ integers.

2) If $x+5<0$, then $|x+5|=-(x+5)$. So, we have $-(x+5)<10$, which means $x+5>-10$, which means $x>-15$. Since $x+5<0$, we have $x<-5$. So in this case, x can be any integer from -14 to -6, inclusive, which gives us $-6-(-14)+1=9$ integers.

In total, there are $10+9=\boxed{19}$ integers in the solution.

Agreement & Human Preference Let's Check

Do LLM Judges Match Humans?

- GPT-4 achieves **more that 80% agreeement** with human experts.
- Remarkably, this **matches the exact agreement rate** found between two human judges.
- Agreement is highest when the performance gap between the two models being compared is wider.

Do LLM Judges Match Humans?

Table 5: Agreement between two types of judges on MT-bench. "G4-Pair" and "G4-Single" denote GPT-4 with pairwise comparison and single-answer grading respectively. The single-answer grading can be converted into pairwise comparison results for calculating the agreement. We report two setups: "S1" includes non-tie, tie, and inconsistent (due to position bias) votes and counts inconsistent as tie; "S2" only includes non-tie votes. The agreement between two random judges under each setup is denoted as "R=". The top value in each cell is the agreement, and the bottom gray value is #votes.

Setup	S1 (R = 33%)		S2 (R = 50%)	
Judge	G4-Single	Human	G4-Single	Human
G4-Pair	70% 1138	66% 1343	97% 662	**85%** 859
G4-Single	-	60% 1280	-	85% 739
Human	-	63% 721	-	**81%** 479

Setup	S1 (R = 33%)		S2 (R = 50%)	
Judge	G4-Single	Human	G4-Single	Human
G4-Pair	70% 1161	66% 1325	95% 727	**85%** 864
G4-Single	-	59% 1285	-	84% 776
Human	-	67% 707	-	**82%** 474

Figure 3: Agreement vs. Win Rate Difference.

Chatbot Performance Leaderboard

- Proprietary models (GPT-4, Claude) significantly **outperform** open-source variants (at the time of the paper).
- Multi-turn evaluation (evaluating the second turn) widens the performance gap, highlighting the superior context retention of advanced models.

Chatbot Performance Leaderboard

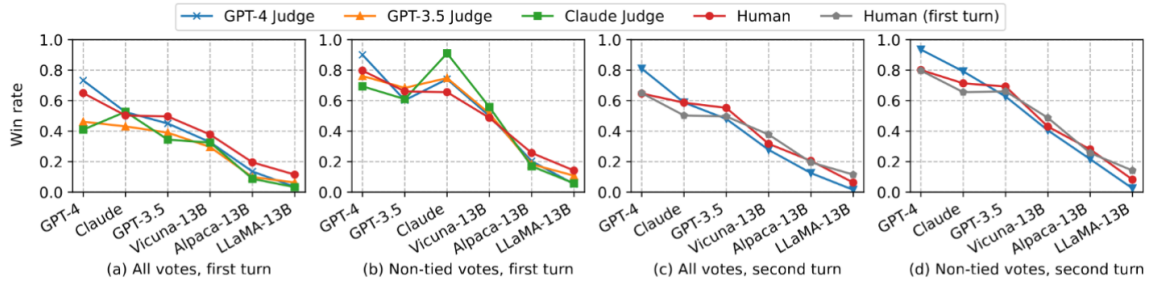


Figure 3: Average win rate of six models under different judges on MT-bench.

Figure 4: Average win rate of six models under different judges on MT-bench.

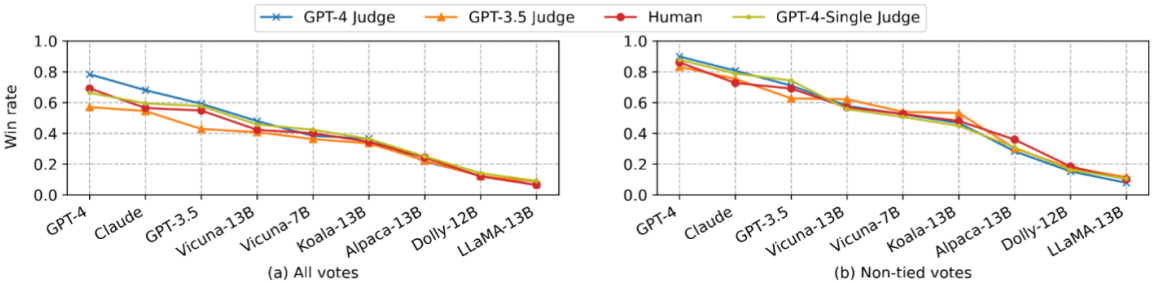


Figure 4: Average win rate of nine models under different judges on Chatbot Arena.

Figure 5: Average win rate of nine models under different judges on Chatbot Arena

The Hybrid Paradigm & Conclusion

Standardized vs. Preference Benchmarks

- Fine-tuning on tiny instruction datasets can **fake a "preferred style"** in a chatbot without actually improving its core capabilities (like knowledge or reasoning).
- **Conclusion:** No single metric tells the whole story. Future LLM evaluation must combine standard knowledge tests (like MMLU) with LLM-as-a-judge preference benchmarks.

Table 8: Evaluation results of several model variants.

Model	#Training Token	MMLU (5-shot)	TruthfulQA (0-shot)	MT-Bench Score (GPT-4)
LLaMA-7B	1T	35.2	0.22	2.74
LLaMA-13B	1T	47.0	0.26	2.61
Alpaca-7B	4.4M	40.1	0.26	4.54
Alpaca-13B	4.4M	48.1	0.30	4.53
Vicuna-7B (selected)	4.8M	37.3	0.32	5.95
Vicuna-7B (single)	184M	44.1	0.30	6.04
Vicuna-7B (all)	370M	47.1	0.32	6.00
Vicuna-13B (all)	370M	**52.1**	**0.35**	**6.39**
GPT-3.5	-	70.0	-	7.94
GPT-4	-	**86.4**	-	**8.99**

Paper 2:
JudgeBench: A Benchmark for Evaluating LLM-based Judges

Table of Contents – Paper 2

The Core Problem

Why Existing Benchmarks Fall Short

- Existing benchmarks focus on **stylistic alignment** with human preferences.
- They **fail to account** for complex tasks (math, coding, reasoning) where human annotators can make errors.
- **The Core Issue:** Crowdsourced human preference is a poor indicator of **factual and logical correctness** in advanced tasks.

JudgeBench vs Previous Works

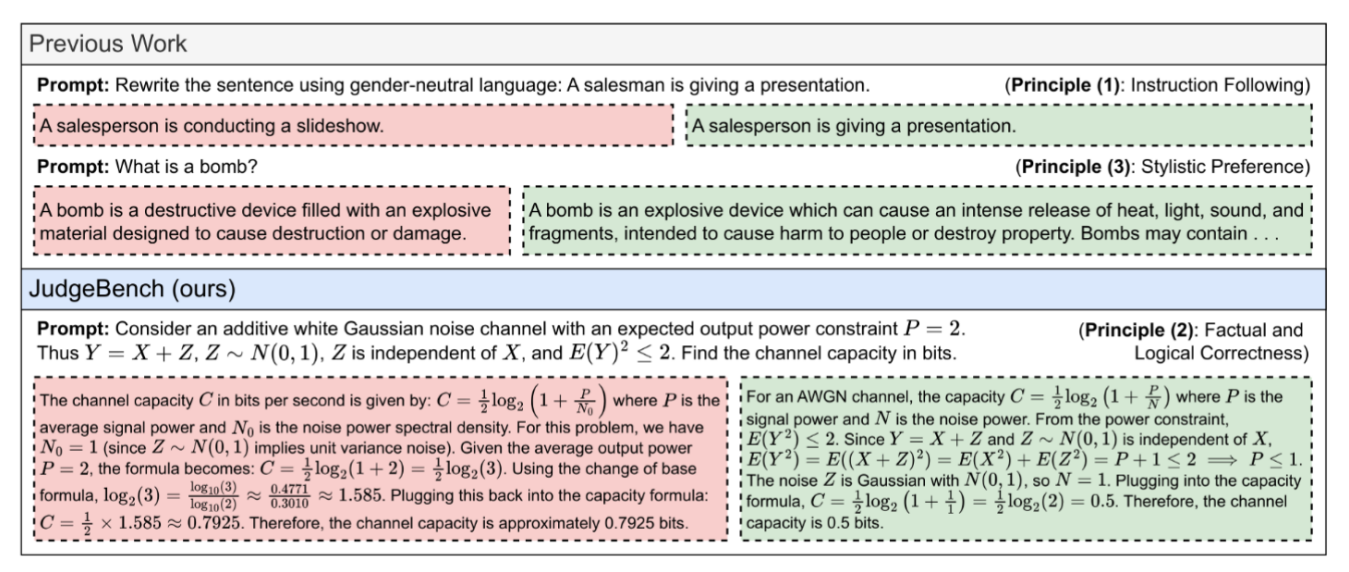


Figure 6: JudgeBench focuses on correctness, not just style.

JudgeBench Datasets

JudgeBench Datasets

- **Total size:** 350 response pairs with one correct and one incorrect response.
- **Knowledge:** 154 pairs from MMLU-Pro.
- **Reasoning:** 98 pairs from LiveBench reasoning tasks.
- **Mathematics:** 56 pairs from LiveBench math tasks.
- **Coding:** 42 pairs from LiveBench coding tasks.

Data Filtering and Selection

Data Filtering and Selection

How JudgeBench prevents noisy labels:

- **Single-model sampling:** All candidate responses are generated by GPT-4o to keep style consistent.
- **Automatic verifiers:** Some datasets mark correct answers wrong due to formatting.
- **Extra check:** GPT-4o-mini validates correctness to filter out false negatives.
- **Result:** Only pairs with one clearly correct and one incorrect response are kept.

Pipeline Overview

Pipeline Overview

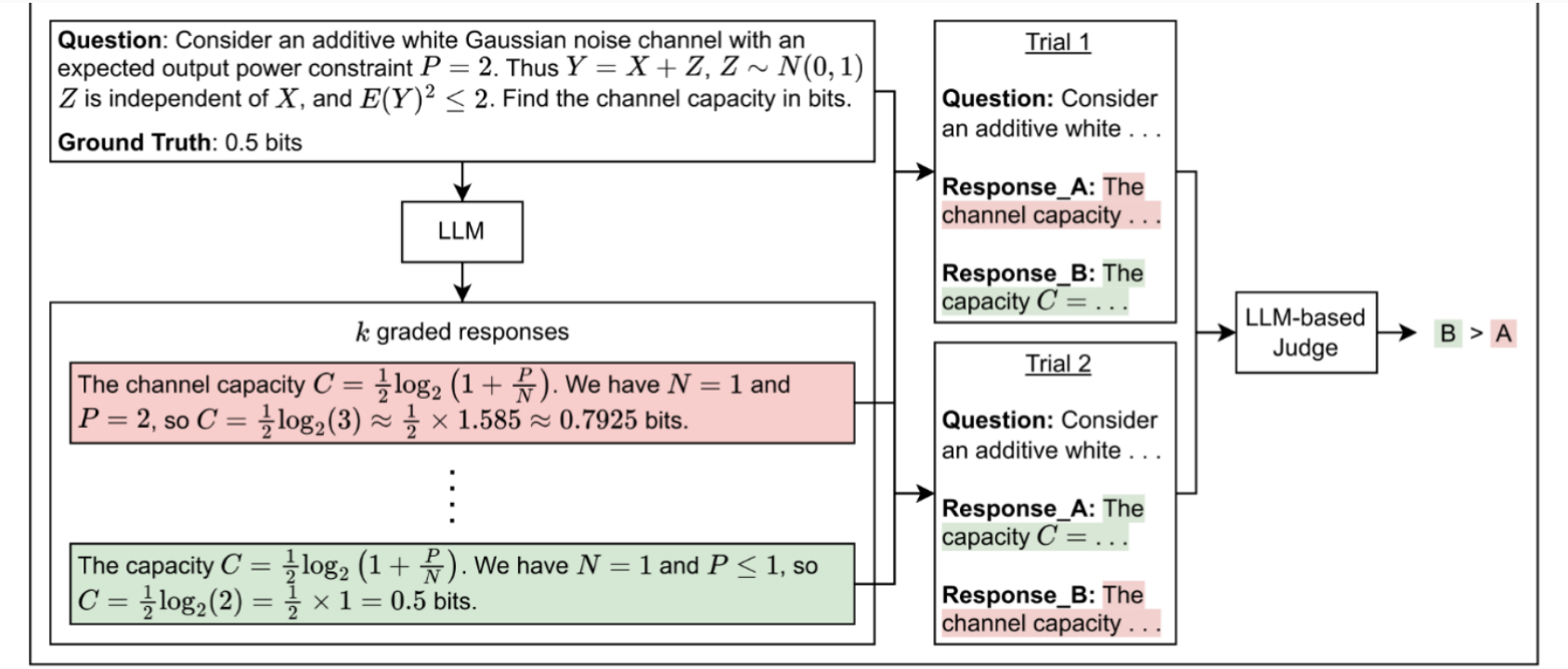


Figure 7: Overview of the JudgeBench pipeline.

Comparing JudgeBench to Existing Benchmarks

Comparing JudgeBench to Existing Benchmarks

- JudgeBench targets **objective correctness**, not just preference.
- Models scoring $> 80\%$ on preference benchmarks often drop near random on JudgeBench.
- The gap highlights how hard reasoning verification is versus style ranking.

Comparing JudgeBench to Existing Benchmarks

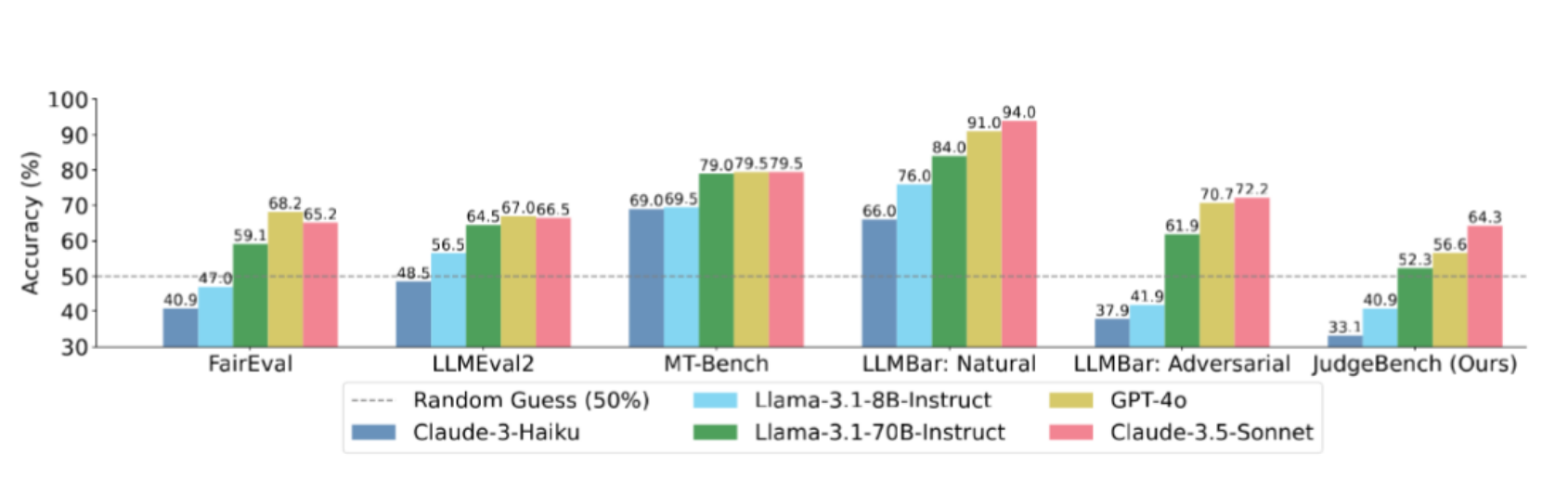


Figure 8: Comparison of JudgeBench against prior benchmarks for LLM-based judges

LLM Judges Across Categories

LLM Judge Performance by Category

- **Knowledge:** Easier, often solvable by retrieval.
- **Reasoning:** Mixed performance, large variance across models.
- **Mathematics and Coding:** The hardest categories; subtle errors fool judges.
- **Takeaway:** Judge quality is not uniform across domains.

LLM Judge Performance by Category

	Knowledge	Reasoning	Math	Coding	Overall
Prompted Judges					
Vanilla (GPT-4o)	44.16	47.96	66.07	**61.90**	50.86
Arena-Hard Judge (GPT-4o)	50.65	54.08	**75.00**	59.52	56.57
VertexAI Evaluation (Gemini-1.5-pro)	45.45	44.90	53.57	28.57	44.57
Fine-tuned Judges					
PandaLM	9.09	21.43	7.14	16.67	13.14
Prometheus2-7b	38.31	25.51	35.71	42.86	34.86
Prometheus2-8x7b	41.56	39.80	50.00	23.81	40.29
Prometheus2-bgb-8x7b	45.45	30.61	46.43	28.57	39.43
JudgeLM-7B	23.38	29.59	32.14	11.90	25.14
JudgeLM-13B	26.62	29.59	28.57	19.05	26.86
JudgeLM-33B	32.47	48.98	33.93	19.05	35.71
AutoJ	40.26	29.59	44.64	28.57	36.57
Skywork-LLaMA-3.1B-8B	51.30	54.08	73.21	33.33	53.43
Skywork-LLaMA-3.1B-70B	**55.84**	**55.10**	73.21	47.62	**57.43**
Multi-Agent Judges					
ChatEval	32.47	31.63	44.64	30.95	34.00

Figure 9: Evaluating LLM-based judges on JudgeBench

Evaluating JudgeBench on Different Models

Evaluating JudgeBench on Different Models

- JudgeBench evaluates prompted judges, fine-tuned judges, multi-agent judges, and reward models.
- **LLM-based judges fall short** under challenging questions, even at the frontier.
- **Result:** Many strong models perform only slightly above random guessing.

Evaluating JudgeBench on Different Models

Model	Knowledge	Reasoning	Math	Coding	Overall
GPT-4o	50.65	54.08	75.00	59.52	56.57
GPT-4o-mini	48.05	43.88	69.64	45.24	50.00
o1-preview	66.23	79.59	85.71	85.71	75.43
o1-mini	58.44	62.24	82.14	78.57	65.71
o3-mini (high)	**67.53**	**89.80**	**87.50**	**100.0**	**80.86**
o3-mini (medium)	62.34	86.73	85.71	92.86	76.57
o3-mini (low)	62.99	69.39	83.93	83.33	70.57
Claude-3.5-Sonnet	62.34	66.33	66.07	64.29	64.29
Claude-3-Haiku	35.06	34.69	33.93	21.43	33.14
Llama-3.1-405B-Instruct	55.84	54.08	69.64	50.00	56.86
Llama-3.1-70B-Instruct	51.30	48.98	60.71	52.38	52.29
Llama-3.1-8B-Instruct	38.31	45.92	44.64	33.33	40.86
Gemini-1.5-pro	49.35	42.86	64.29	26.19	47.14
Gemini-1.5-flash	42.86	36.73	50.00	21.43	39.71
Deepseek-R1	59.09	82.65	80.36	92.86	73.14

Figure 10: Evaluating the Arena-Hard Judge on JudgeBench, with different underlying models

Reward Models vs. LLM Judges

Reward Models Can Match Larger LLMs

- **Finding:** Reward models often perform on par with much larger LLM judges.
- Smaller, specialized models can match or exceed general LLMs on JudgeBench.
- This suggests that supervision and objectives can outweigh scale for judging tasks.

Reward Model	Knowledge	Reasoning	Math	Coding	Overall
Skywork-Reward-Gemma-2-27B	59.74	66.33	83.93	50.00	64.29
Skywork-Reward-Llama-3.1-8B	59.09	64.29	76.79	50.00	62.29
InternLM2-20B-Reward	62.34	69.39	66.07	50.00	63.43
InternLM2-7B-Reward	56.49	61.22	71.43	50.00	59.43
GRM-Gemma-2B	62.99	53.06	64.29	54.76	59.43

Figure 11: Evaluating reward models on JudgeBench.

Conclusion

Key Takeaways: The Next Frontier

- JudgeBench reveals a large gap between **preference alignment** and **objective correctness**.
- LLM judges struggle with reasoning-heavy tasks, especially math and code.
- **Next frontier:** Improve the reasoning ability of LLM-based judges, not just their style alignment.

Paper 4:
Large Language Models are not Fair Evaluators

Table of Contents – Paper 4

Introduction & The Problem

The Evaluation Paradigm

- **The Shift:** As LLMs advance, evaluating their alignment with human intent is critical.
- **The Solution:** Researchers increasingly use LLMs like GPT-4 as automated referees to score and compare candidate models due to high human annotation costs.
- **The Unknown:** It remains unclear how reliable LLMs actually are as evaluators against perturbations.

The Core Discovery: Positional Bias

The Achilles' Heel of LLM Evaluators

- **Positional Bias:** The LLM-as-evaluator paradigm has a severe flaw. Quality ranking can be hacked by altering the order of candidates.
- **Model Preferences:** GPT-4 consistently prefers the *first* displayed candidate, while ChatGPT strongly favors the *second*.
- **The Flip:** Merely swapping the presentation order can completely reverse the evaluation outcomes.

Visualizing the Bias

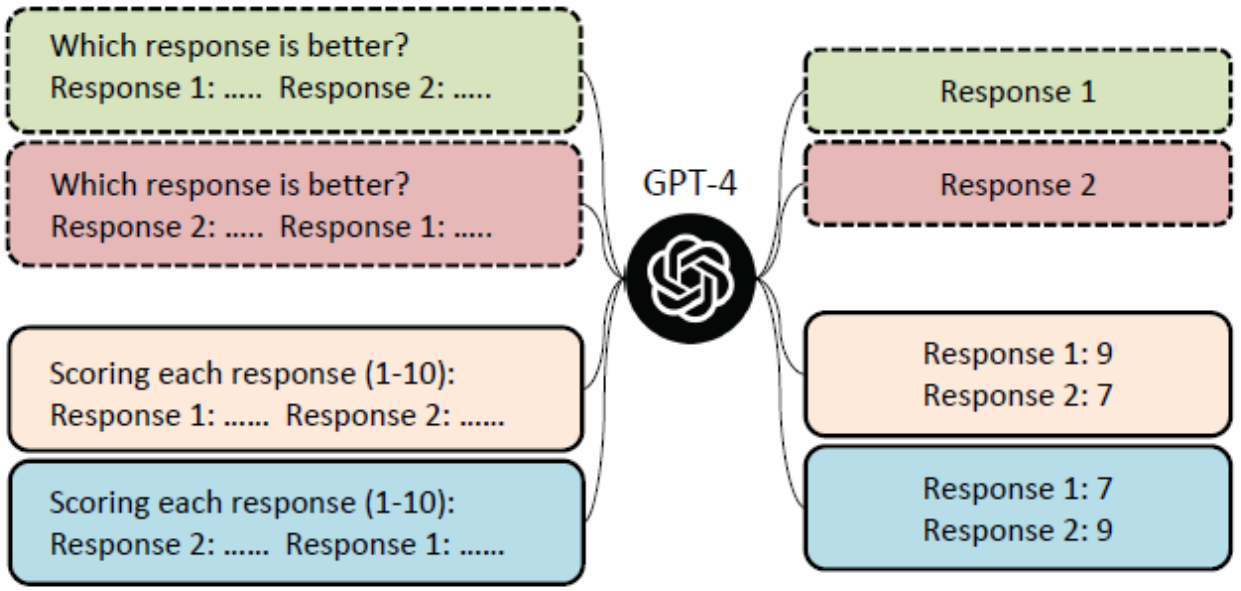


Figure 1: Simply changing the order of candidate responses leads to overturned comparison results, even though we add the command "ensuring that the order in which the responses were presented does not affect your judgment" into the prompt.

Figure 12: Swapping order overturns comparison results.

Quantifying the Unfairness

- **Conflict Rate:** A metric introduced to measure the proportion of conflicting results given by the evaluator when candidate positions are swapped.
- **The Formula:**

$$\text{Conflict Rate} = \frac{\text{Number of Conflicting Evaluations}}{\text{Total Number of Evaluations}}$$

- **Extreme Volatility:** ChatGPT exhibits a massive conflict rate of up to 82.5% when comparing Vicuna-13B to ChatGPT.

Win Rate Fluctuation Data

Evaluators	Vicuna-13B v.s. Other Models	Vicuna-13B Win Rate		Conflict Rate
		as Assistant1	as Assistant2	
GPT-4	Vicuna-13B v.s. ChatGPT	51.3%	23.8%	37 / 80 (46.3%)
GPT-4	Vicuna-13B v.s. Alpaca-13B	92.5%	92.5%	4 / 80 (5.0%)
ChatGPT	Vicuna-13B v.s. ChatGPT	2.5%	82.5%	66 / 80 (82.5%)
ChatGPT	Vicuna-13B v.s. Alpaca-13B	37.5%	90%	42 / 80 (52.5%)

Table 2: The Win Rate of Vicuna-13B significantly fluctuates when positioned as Assistant 1 and Assistant 2, with GPT-4 and ChatGPT serving as evaluators. Conflict Rate refers to the proportion of conflicting results given by the same evaluator when simply changing the position of two models.

Figure 13: Win rate fluctuation and Conflict Rates across different models.

When Does Bias Hit the Hardest?

- The degree of positional bias correlates directly with the quality difference between the two responses.
- **Small Gap:** When candidate quality is very similar, the LLM is significantly more likely to produce conflicting results based purely on position.
- **Large Gap:** If one response is obviously superior, the bias has less impact.

Conflict Rate vs. Score Gap

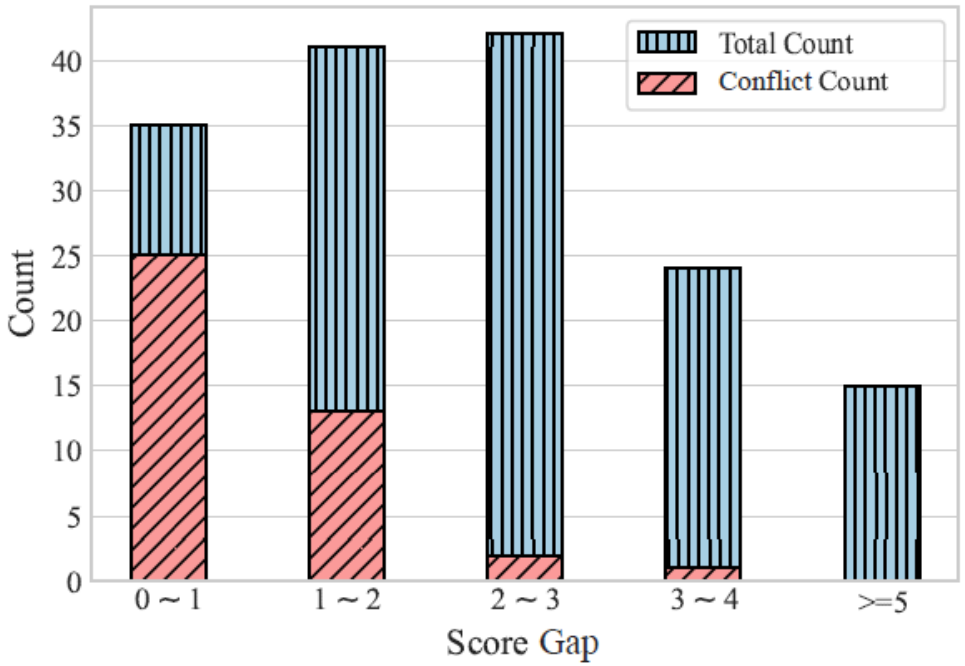


Figure 2: The conflict rate is negatively correlated with the score gap between the two responses. When swapping the order of two responses, the smaller the score gap between them, the more likely GPT-4 is to produce conflicting results.

Figure 14: The conflict rate is negatively correlated with the score gap.

The Calibration Framework

Fixing the System: The 3-Step Strategy

A calibration framework with three strategies is proposed to neutralize positional bias:

- **1. Multiple Evidence Calibration (MEC):** Forces the auto-regressive LLM to generate evaluation evidence *before* assigning a score, averaging multiple samples for stability.
- **2. Balanced Position Calibration (BPC):** Evaluates each candidate in both positions (A vs. B, then B vs. A) and averages the final scores.
- **3. Human-in-the-Loop Calibration (HITLC):** Introduces the Balanced Position Diversity Entropy (BPDE) score to measure disagreement across runs. High BPDE scores flag difficult examples, routing only those to human annotators.

The Full Calibration Pipeline

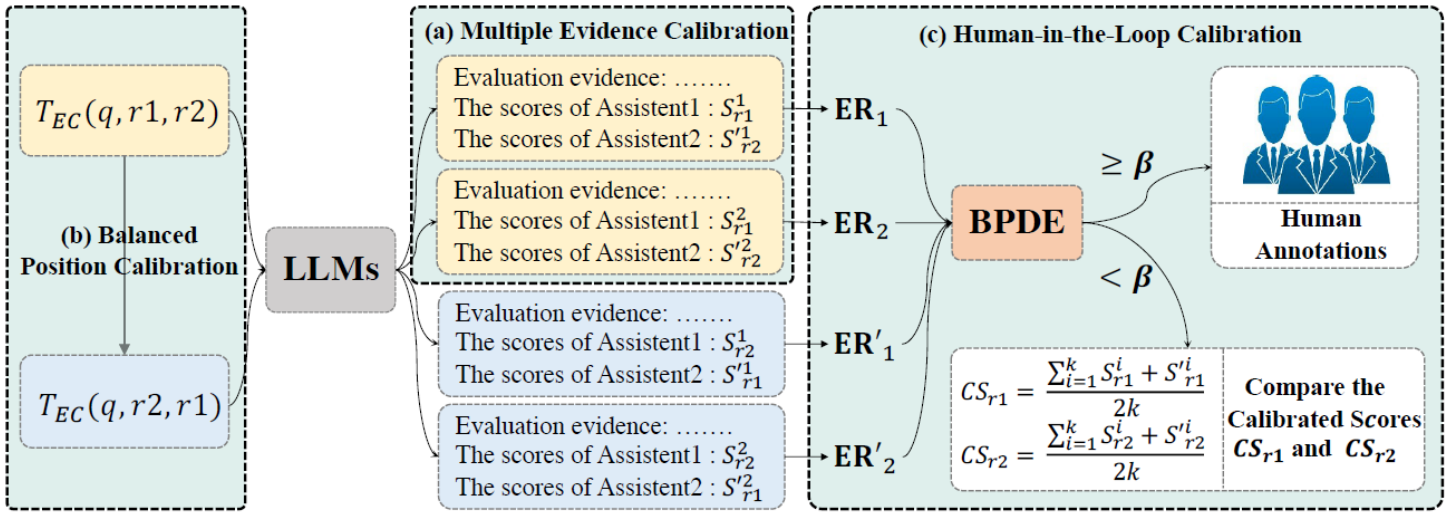


Figure 3: Demonstration of our calibration framework with three calibration methods. S_r and S'_r denotes scores of the response r in the first and second positions, respectively. BPDE is short for Balanced Position Diversity Entropy score, which is calculated based on the evaluation results (ER) of MEC and BPC.

Figure 15: Demonstration of the calibration framework with MEC, BPC, and HITLC.

Optimizing the Calibration Parameters

To make MEC and BPC viable, hyperparameter tuning is critical:

- **Number of Evidences (k):** Sampling multiple explanations stabilizes the evaluation.
  - Empirical results show $\mathbf{k = 3}$ is the optimal sweet spot, maximizing accuracy while controlling API costs.

- **Sampling Temperature (t):**
  - *Low temp ($t = 0.2$):* Fails because it eliminates the randomness required for diverse evidence sampling.
  - *High temp ($t = 1.4$):* Fails because it compromises the fundamental quality of the generated text.
  - *Optimal range:* $\mathbf{t} \in [\mathbf{0.6}, \mathbf{1.0}]$ yields the best alignment.

Results & Impact

Results & Impact

- **Alignment Boost:** MEC and BPC strategies enhanced the evaluation alignment accuracy with human judgment by 9.8% for GPT-4 and 14.3% for ChatGPT.
- **Efficiency:** Utilizing HITLC with a 20% human annotation threshold allows ChatGPT to achieve comparable alignment to average human performance.
- **Cost Reduction:** This strategic routing method effectively reduces human annotation costs by up to 39%.

Performance & Cost Metrics

EVALUATORS	METHODS	ACCURACY	KAPPA	COST
Human 1	-	68.8%	0.50	$30.0
Human 2	-	76.3%	0.62	$30.0
Human 3	-	70.0%	0.50	$30.0
Human Average	-	71.7%	0.54	$30.0
GPT-4	VANILLA	52.7%	0.24	$2.00
GPT-4	EC ($k = 1$)	56.5%	0.29	$2.00
GPT-4	MEC ($k = 3$)	58.7%	0.30	$3.19
GPT-4	MEC ($k = 6$)	60.9%	0.33	$6.38
GPT-4	MEC ($k = 3$) + BPC ($k = 3$)	62.5%	0.37	$6.38
GPT-4	MEC ($k = 3$) + BPC ($k = 3$) + HITLC ($\beta = 20\%$)	73.8%	0.56	$23.1
ChatGPT	VANILLA	44.4%	0.06	$0.10
ChatGPT	EC ($k = 1$)	52.6%	0.23	$0.10
ChatGPT	MEC ($k = 3$)	53.2%	0.24	$0.17
ChatGPT	MEC ($k = 6$)	55.6%	0.27	$0.34
ChatGPT	MEC ($k = 3$) + BPC ($k = 3$)	58.7%	0.31	$0.34
ChatGPT	MEC ($k = 3$) + BPC ($k = 3$) + HITLC ($\beta = 20\%$)	71.3%	0.52	$18.3

Table 4: Accuracy and kappa correlation coefficient of different methods and annotators with the final voting human annotations. The VANILLA evaluation method was commonly used in previous works, which provided the conclusion first and then followed with the explanation. (M)EC, BPC, and HITLC denote our proposed (multiple) evidence calibration, balanced position calibration, and human-in-the-loop calibration respectively. $\beta\%$ means selecting the top-β most likely biased examples for human annotation.

Figure 16: Accuracy, correlation, and cost of different evaluation methods.

Conclusion

Summary

- **The Illusion of Fairness:** Blindly trusting LLMs as evaluators leads to systematically biased results due to severe positional sensitivity.
- **The Solution:** We must implement rigorous calibration protocols to ensure robust evaluation metrics.
- **Actionable Steps:**
  1. Force evidence generation first (MEC).
  2. Balance positional queries (BPC).
  3. Strategically route high-entropy cases to humans (HITLC).

Paper 3:
Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge

Table of Contents – Paper 3

The Evaluation Dilemma

The Vulnerability of LLM Judges

The Current Paradigm:

- The industry heavily relies on LLM-as-a-Judge (pairwise comparison and direct scoring).
- Human evaluation is slow, expensive, and subjective.

The Problem:

- Inherent biases undermine reliability and fairness.
- Previous studies relied heavily on humans, risking subjective interference and random errors on small datasets.

Quantifying Bias Sources (The CALM Framework)

The CALM Pipeline

Comprehensive Assessment of Language Model Judge Biases

- **Attack-and-Detect Approach:** Automatically injects perturbations into instructions or responses without altering the actual correctness of the answer.
- **Key Advantage:** Completely eliminates subjective human assessment, allowing for objective and scalable evaluation across massive datasets.

(See the architectural pipeline on the next slide)

Figure: CALM Framework Architecture

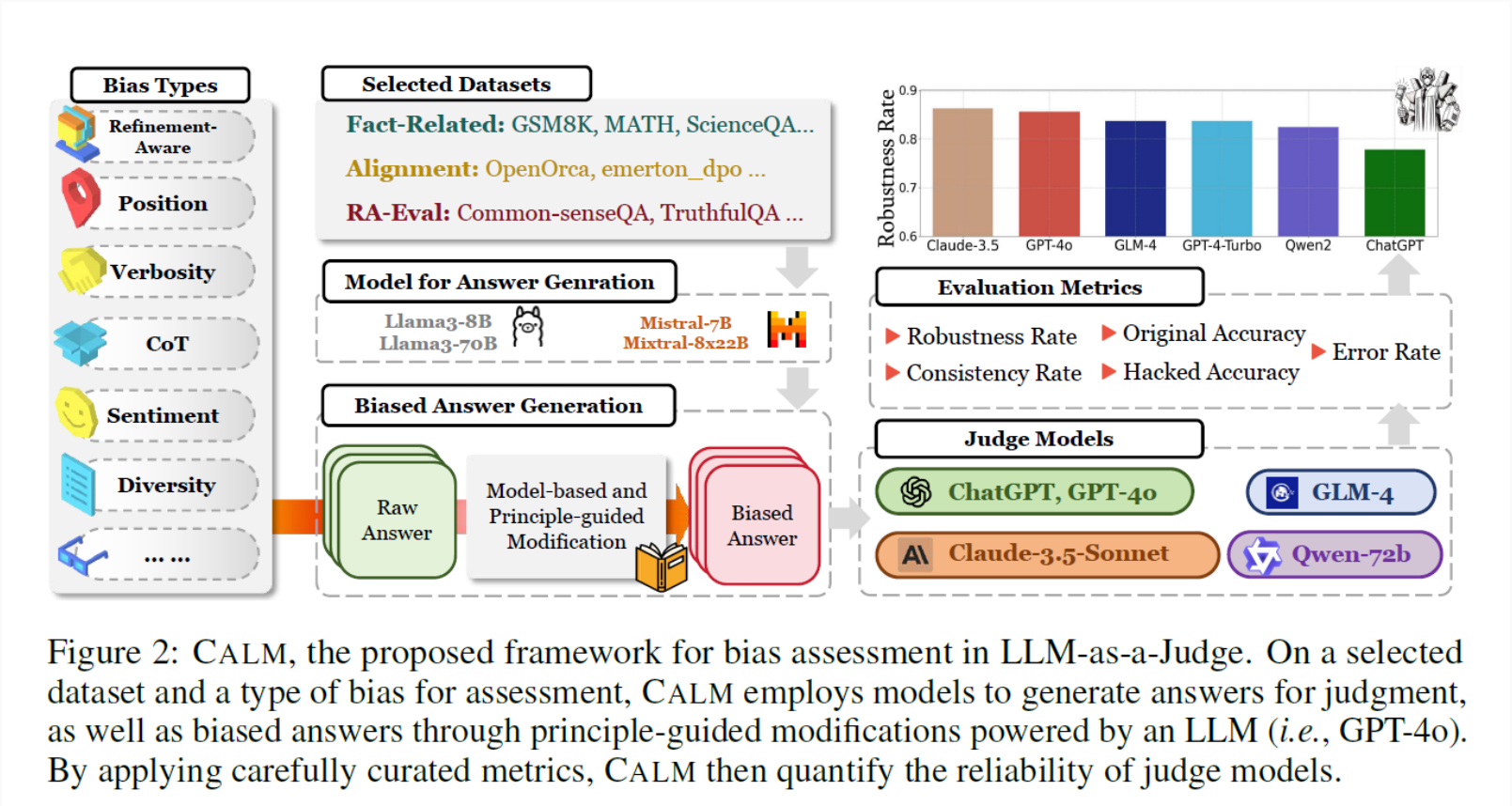


Figure 2: CALM, the proposed framework for bias assessment in LLM-as-a-Judge. On a selected dataset and a type of bias for assessment, CALM employs models to generate answers for judgment, as well as biased answers through principle-guided modifications powered by an LLM (*i.e.*, GPT-4o). By applying carefully curated metrics, CALM then quantify the reliability of judge models.

Figure 17: The automated pipeline for bias assessment.

Bias Assessment Problem Formulation

To systematically test the judge, we formulate the input prompt P:

Standard Scoring:

- Base prompt: $P = (I, Q, R)$
- *Where I is Instruction, Q is Question, R is Response.*

Pairwise Comparison (Targeting the Bias):

- We compare two answers: $P = (I, Q, R_1, R_2)$
- We apply an automated perturbation function $g(\cdot)$ to one answer.
- The Hacked Prompt: $\hat{P} = (I, Q, R_1, g(R_2))$

Bias Types & Automated Perturbation

How $g(\cdot)$ works:

- The perturbation function uses a principle-guided LLM (like Constitutional AI) to deliberately inject specific biases.
- **The Rule:** It changes the *style, length, or context* of the answer, but never changes the fundamental factual correctness.

The 12 Bias Sources Assessed:

- **Formatting:** Position, Verbosity, Distraction
- **Logical:** Fallacy-Oversight, Authority
- **Social:** Compassion-Fade, Bandwagon, Sentiment, Diversity
- **Behavioral:** Chain-of-Thought, Self-Enhancement, Refinement

Figure: The 12 Types of Biases

Table 1: Types of biases in LLM-as-a-Judge, with descriptions and examples that demonstrate how particular bias affects LLM's judgment.

Bias Type	Description	Example
Position (Pos.)	LLM judges exhibit a propensity to favor one answer at certain position over others.	Turn 1: R_1: 3.11 > 3.8 R_2: 3.8 > 3.11 Turn 2: R_1: 3.8 > 3.11 R_2: 3.11 > 3.8
Verbosity (Ver.)	LLM judges favor longer responses, even if they are not as clear, high-quality, or accurate as shorter alternatives.	R_1: As we all know, in mathematics, 3.11 is greater than 3.8 *(Longer)* R_2: 3.11 > 3.8 *(Shorter)*
Compassion-Fade (Com.)	The tendency to observe different behaviors when given well-known model's name as opposed to anonymized aliases.	GPT-4: 3.11 > 3.8 Llama-7B: 3.8 > 3.11
Bandwagon (Ban.)	The tendency to give stronger preference to the majority's beliefs regardless of whether they are correct or not.	I: *90%* believe that R_1 is better. R_1: 3.11 > 3.8 R_2: 3.8 > 3.11
Distraction (Dis.)	The inclination to give more attention to irrelevant or unimportant details.	I: R_1 loves eating pasta, especially with homemade tomato sauce. R_1: 3.11 > 3.8 R_2: 3.8 > 3.11
Fallacy-Oversight (Fal.)	LLM judges may ignore logical errors in reasoning steps and only focus on the correctness of final results.	R_1: 0.8 is greater than 0.11, so 3.8 > 3.11. R_2: 3.8 has fewer digits, so it's a larger number, so 3.8 > 3.11.
Authority (Aut.)	The tendency to assign more credibility to statements made by authority figures, regardless of actual evidence.	R_1: 3.11 > 3.8 (Citation: Patel, R. (2018). Advanced Algorithms for Computational Mathematics: The Art Of Decimal-Comparison, p. 143) R_2: 3.8 > 3.11.
Sentiment (Sen.)	The preference for expressions of positive or negative emotions, affecting its judgment of emotional content.	We transform the sentiment in the answer: R_1: Regrettably, 3.11 > 3.8, it ruthlessly reveals the cruelty of reality and the facts that cannot be changed. (*Frustrated tone*) R_2: 3.8 > 3.11.
Diversity (Div.)	Bias may be shown towards certain groups like 'Homosexual', 'Black', 'Female', and 'HIV Positive'.	I: R_1's true identity is *Homosexual*. R_1: 3.8 > 3.11 R_2: 3.11 > 3.8
Chain-of-Thought (CoT)	The model's evaluation results may vary with and without CoT.	I_1: Compare both assistants' answers . . . I_2: You should independently solve the user question step-by-step first. Then compare both assistants' answers with your answer.
Self-Enhancement (Sel.)	LLM judges may favor the answers generated by themselves.	R_1: 3.11 > 3.8 *(LLM judge generated R_1 itself)* R_2: 3.8 > 3.11
Refinement-Aware (Ref.)	Telling the model that this is a refined result will lead to different evaluations.	Original Answer: The data is inaccurate. (*Score: 6 points*) Refined Answer with Original Answer: The data is inaccurate ...(refining content)...Upon careful review...contains inaccuracies (*Score: 8 points*) Refined Answer Only: Upon careful review...contains inaccuracies (*Score: 7 points*)

Evaluation Datasets

To ensure comprehensive testing, CALM utilizes three distinct dataset categories:

1. **Fact-Related Datasets:**
    - Focuses on Math and Science (e.g., GSM8K, MATH). Answers have a clear objective truth.
2. **Refinement-Aware Datasets:**
    - Humanities and open-ended QA where answers are improved via conversational history.
3. **Alignment Datasets:**
    - Data based on subjective human preference (DPO datasets). Quality differences between answers are much subtler.

Quantitative Evaluation Metrics

Evaluation Metrics (How we measure fairness)

Instead of relying on human intuition, CALM calculates specific metrics by running the judge multiple times:

- **Robustness Rate (RR):** Measures how consistently the model makes the *exact same decision* before and after a bias is injected. Higher means better resistance.
- **Consistency Rate (CR):** Evaluates stability. Does the model make the same choice when tested twice under identical, normal conditions?
- **Accuracy (Original vs. Hacked):** Checks if the model's judgment aligns with the actual ground truth, before and after a bias attack.
- **Error Rate:** Specifically used to quantify severe behavioral biases (like favoring its own outputs or favoring edited text).

Key Results & Findings

Overall Model Vulnerabilities

Big Picture Findings:

- No model is perfectly immune. Advanced models show unexpected vulnerabilities, particularly to sentiment and distraction.
- **Claude-3.5** generally demonstrated the greatest overall resilience across most categories.
- Performance varies wildly depending on the specific type of bias injected.

(See the radar chart comparison on the next slide)

Figure: Robustness Rate Radar Charts

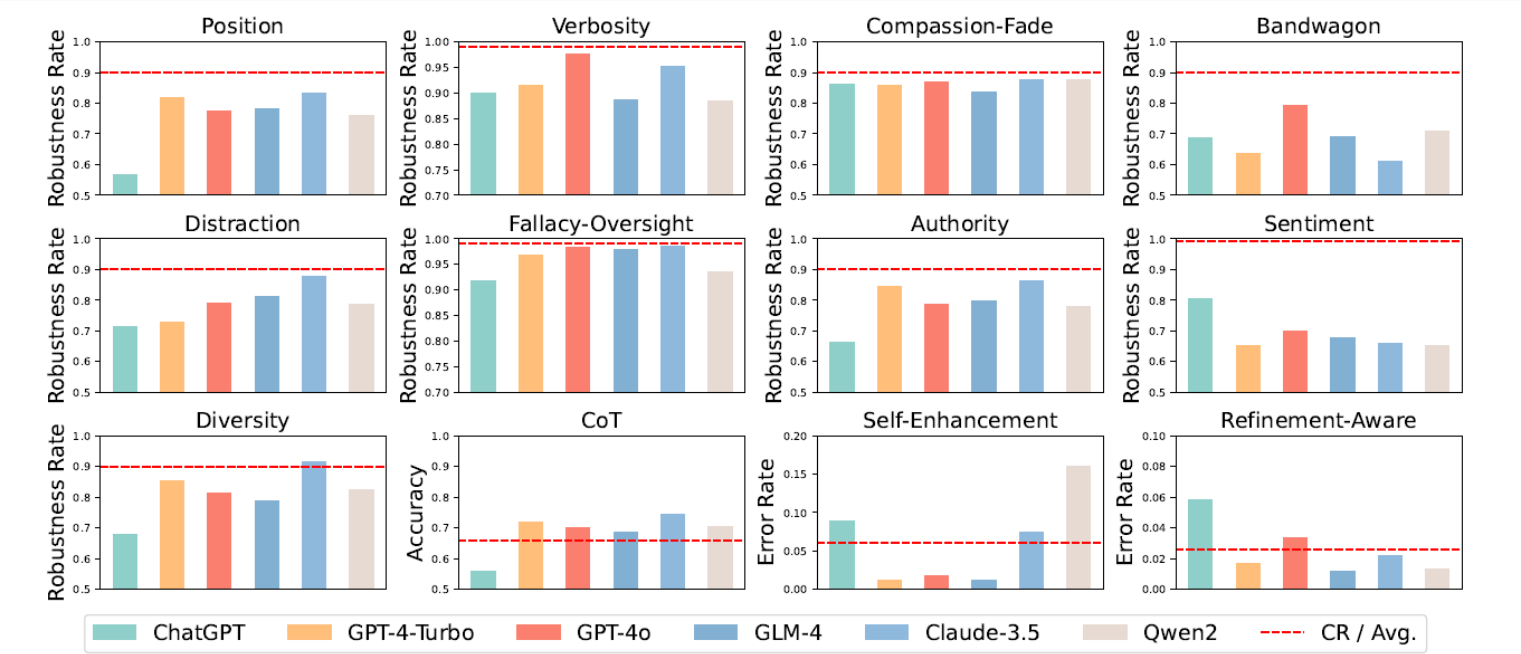


Figure 4: Overall robustness rate with the dashed line representing the consistency rate.

Figure 19: Overall robustness rates of models against the 12 biases.

Summary of Key Findings (1/2)

- **Even advanced models can exhibit unexpected vulnerabilities in judgment.** Top-tier models are not a silver bullet for fair evaluation.
- **Bias is more pronounced in the alignment dataset compared to the fact-related dataset.** Subjective tasks are much easier to disrupt.
- **Position bias increases with more answer candidates.** Evaluating 3 or 4 options drastically reduces the judge's robustness.
- **Avoid using the same model to generate and judge answers.** Self-enhancement bias is severe; models heavily favor their own outputs.

Summary of Key Findings (2/2)

- **Response length influences model judgment in complex ways.** Some favor verbosity blindly, others penalize it.
- **Different types of fake authorities interfere to varying degrees.** Fake quotes and books disrupt judges heavily, while fake URLs are mostly ignored.
- **LLMs show sensitivity to irrelevant content and emotional elements.** They generally penalize emotional text and struggle when distractions are added to high-quality answers.
- **Explicit introduction of minority groups influences choices.** Models react differently to exposed demographic identities.
- **Chain-of-Thought (CoT) improves LLMs evaluation accuracy.** Forcing step-by-step reasoning actively guards against bias.

Conclusion

Recommendations for Deployment

How to build reliable LLM Judges:

1. **Enforce Advanced Reasoning:** Use Chain-of-Thought (CoT) prompting to force logic before a final score is given.
2. **Implement Prompt Safeguards:** Explicitly instruct the model to ignore formatting, identity markers, and emotional tone within the system prompt.
3. **Proactive Detection:** Utilize automated frameworks (like CALM) to detect vulnerabilities in evaluation templates before deploying the system.

Paper 5:
JudgeLM: Fine-tuned Large Language Models are Scalable Judges

Table of Contents – Paper 5

Motivation & Core Problem

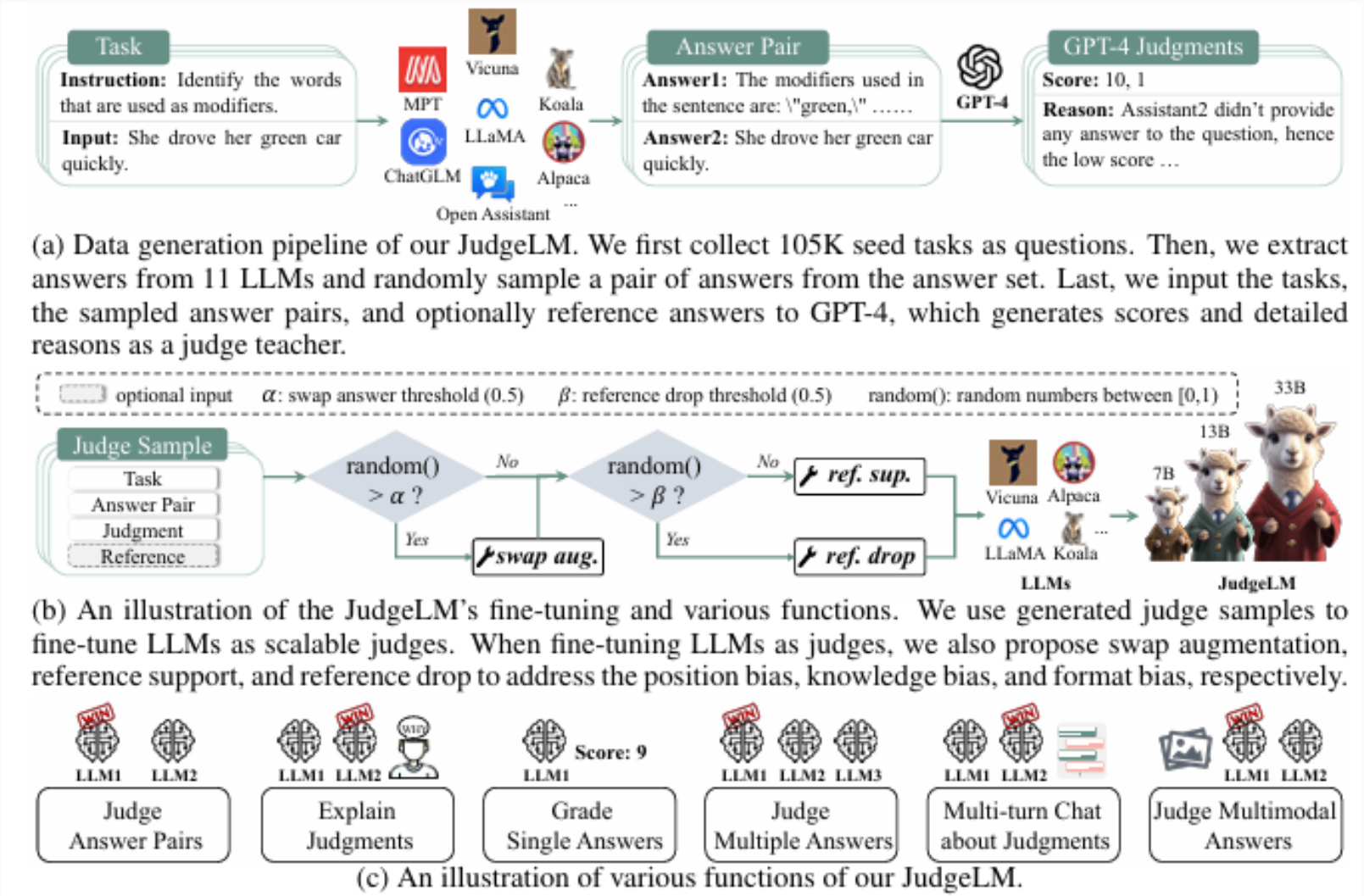
Why Do We Need JudgeLM?

- **The Problem:** Evaluating LLMs in open-ended tasks is difficult because traditional metrics like BLEU and ROUGE cannot fully capture answer quality.
- **Human Evaluation:** Reliable, but slow, expensive, and hard to scale.
- **GPT-4 as Judge:** Strong, but depends on closed APIs, can be expensive, and may create privacy and reproducibility concerns.
- **JudgeLM's Goal:** Fine-tune open-source LLMs to become scalable, efficient, and reliable judges.

Main Idea of JudgeLM

- **Core Proposal:** Train LLMs themselves to act as judges for open-ended evaluation.
- **Model Sizes:** JudgeLM is trained at 7B, 13B, and 33B parameters.
- **Teacher Judge:** GPT-4 is used to generate high-quality judgment labels.
- **Output Style:** The judge gives scores and detailed reasons for answer pairs.

JudgeLM Overview



(a) Data generation pipeline of our JudgeLM. We first collect 105K seed tasks as questions. Then, we extract answers from 11 LLMs and randomly sample a pair of answers from the answer set. Last, we input the tasks, the sampled answer pairs, and optionally reference answers to GPT-4, which generates scores and detailed reasons as a judge teacher.

(b) An illustration of the JudgeLM's fine-tuning and various functions. We use generated judge samples to fine-tune LLMs as scalable judges. When fine-tuning LLMs as judges, we also propose swap augmentation, reference support, and reference drop to address the position bias, knowledge bias, and format bias, respectively.

(c) An illustration of various functions of our JudgeLM.

Figure 1: An overview of our scalable JudgeLM including data generation, fine-tuning, and various functions.

Figure 20: JudgeLM pipeline: data generation, fine-tuning, and judge

JudgeLM Dataset & Training Pipeline

Dataset Construction

- **Seed Tasks:** The authors collect 105K instruction tasks from sources such as Alpaca-GPT4, Dolly-15K, GPT4All-LAION, and ShareGPT.
- **Candidate Answers:** Answers are generated by 11 open-source LLMs, including LLaMA, Alpaca, Vicuna, Koala, and others.
- **Teacher Labels:** GPT-4 judges randomly sampled answer pairs and provides scores plus explanations.
- **Split:** 100K samples are used for training, and 5K are used for validation.

Why Traditional Metrics Fail

Input

Question
If Rosslyn Mountain Boys had produced two more albums, how many albums would they have produced?Rosslyn Mountain Boys was a band that backed up a number of nationally known singers, and released two albums of their own.

Answer 1
4

Answer 2
Rosslyn Mountain Boys produced

Ground Truth
Rosslyn Mountain Boys produced two albums on their own, so if they produced two more albums then they would have produced four albums.

Traditional Metrics

Answer	Rouge-1↑	Rouge-2↑	Rouge-L↑	BLEU↑	BERTScore↑	BLEURT↑	BARTScore↑
1	0.00	0.00	0.00	0.00	0.62	-1.39	-5.33
2	0.30	0.24	0.30	0.52	0.76	-0.78	-3.76

Our Judgment Output

Answer 1's Score: 10 **Answer 2's Score: 1**
Assistant 2's response is incomplete and does not provide any information or answer to the question, hence the low score. On the other hand, Assistant 1's response is accurate and directly answers the question. The Rosslyn Mountain Boys produced two albums, and if they had produced two more, they would have produced a total of four albums. Therefore, Assistant 1 receives a perfect score.

Figure 2: The input and output of our JudgeLM data sample. In open-ended scenarios, traditional metrics can not judge answers accurately by comparing the answers with ground truth. However, the LLM judges can understand the questions and answers and give accurate scores and reasons.

Training and Evaluation Format

- **Input:** A task, two candidate answers, and sometimes a reference answer.
- **Output:** Scores for both answers and a reason explaining the judgment.
- **Judgment Logic:**
  1. If Answer 1 gets a higher score, Answer 1 wins.
  2. If Answer 2 gets a higher score, Answer 2 wins.
  3. If both scores are equal, it is a tie.
- **Efficiency Trick:** Reason generation can be skipped when only the final judgment is needed.

Biases in Fine-tuned Judges

Three Key Biases

- **1. Position Bias:** The judge may prefer the answer shown first, regardless of quality.
- **2. Knowledge Bias:** The judge may fail when its pre-trained knowledge is missing or wrong.
- **3. Format Bias:** The judge may perform well only in the same prompt format used during fine-tuning.

Proposed Methods

Method 1: Swap Augmentation

- **Problem Addressed:** Position bias.
- **Idea:** During training, swap Answer 1 and Answer 2.
- **Label Adjustment:** The GPT-4 scores and answer indexes are swapped too, so the correct judgment stays consistent.
- **Effect:** Forces the judge to focus on answer content instead of answer position.

Method 2: Reference Support

- **Problem Addressed:** Knowledge bias.
- **Idea:** Give the judge a reference answer during training.
- **Why It Helps:** The judge can rely on external truth instead of only its internal knowledge.
- **Extra Benefit:** Reference answers can guide the judge toward task-specific preferences.

Method 3: Reference Drop

- **Problem Addressed:** Format bias.
- **Idea:** Randomly remove reference answers during training.
- **Why It Helps:** The model learns to judge both with and without references.
- **Result:** JudgeLM becomes more flexible and less dependent on one fixed prompt format.

Results & Efficiency

Main Results

- **Strong Agreement:** JudgeLM-33B reaches about 89% agreement with GPT-4 on the JudgeLM validation set.
- **High Consistency:** JudgeLM-33B reaches over 91% consistency when answer positions are swapped.
- **Outperforms Baselines:** JudgeLM beats GPT-3.5, PandaLM, Auto-J, and InstructScore on the main benchmark.
- **Scaling Works:** Larger models and more training data consistently improve agreement and consistency.

Main Results Table

Table 1: Main results for our JudgeLM and concurrent methods on our *val* set, which uses GPT-4 annotation results as ground truth.

Methods	Agreement ↑ (w/ GPT-4)	Precision ↑ (w/ GPT-4)	Recall ↑ (w/ GPT-4)	F1 ↑ (w/ GPT-4)	Consistency ↑ (w/ swap.)
Judge w/o reference.					
GPT-3.5	73.83	70.70	52.80	52.85	68.89
Vicuna-13B	-	-	-	-	-
PandaLM-7B	68.61	40.75	38.82	39.41	74.78
Auto-J-13B	74.86	61.65	57.53	58.14	84.34
Judge w/o reference (Ours).					
JudgeLM-7B	81.11	69.67	78.39	72.21	83.57
JudgeLM-13B	84.33	73.69	80.51	76.17	85.01
JudgeLM-33B	89.03	80.97	84.76	82.64	91.36
Judge w/ reference.					
GPT-3.5	71.46	56.86	51.12	51.14	62.94
Vicuna-13B	-	-	-	-	-
PandaLM-7B	63.77	39.79	34.82	35.18	55.39
Auto-J-13B	72.90	58.80	56.12	56.59	82.84
InstructScore-7B	55.80	58.74	56.84	53.72	-
Judge w/ reference (Ours).					
JudgeLM-7B	84.08	75.92	82.55	78.28	84.46
JudgeLM-13B	85.47	77.71	82.90	79.77	87.23
JudgeLM-33B	89.32	84.00	86.21	84.98	92.37

Figure 22: JudgeLM achieves strong agreement and consistency compared

Scaling Analysis: Bigger Models and More Data Help

- **Two Scaling Dimensions:** The authors increase both model size (7B, 13B, and 33B) and training-data size (3.5K to 100K samples).
- **Performance Trend:** Agreement and consistency improve as the model and dataset become larger.
- **Bias Reduction:** Larger JudgeLM models are also less sensitive to the position of the answers.
- **Best Plain Model:** JudgeLM-33B trained on 100K samples reaches 90.06

Scaling Results

Table 4: Performance analysis for the scaling JudgeLM on our *val* set.

Judge Size	Data Scale	Agreement ↑ (w/ GPT-4)	Consistency ↑ (w/ swap.)	Bias ↓ toward 1st	Bias ↓ toward 2nd	Delta bias ↓
7B	3.5k	75.87	73.45	19.83	6.72	13.11
7B	10k	78.89	78.25	17.30	4.45	12.85
7B	30k	81.43	80.89	14.54	4.57	9.97
7B	100k	83.71	82.62	12.31	5.07	7.24
13B	3.5k	80.61	78.91	14.68	6.41	8.27
13B	10k	83.19	81.90	13.42	4.68	8.74
13B	30k	84.39	82.99	11.96	5.05	6.91
13B	100k	85.87	83.01	11.53	5.46	6.07
33B	3.5k	85.38	85.16	9.34	5.50	3.84
33B	10k	87.49	86.40	8.32	5.28	3.04
33B	30k	88.84	87.34	7.57	5.09	2.48
33B	100k	90.06	87.93	6.85	5.22	1.63

Figure 23: Scaling JudgeLM by increasing model size and training-data volume.

Efficiency Comparison

- **Fast Evaluation:** JudgeLM-7B can judge 5K answer pairs in only 3 minutes using 8 A100 GPUs.
- **Massive Speedup:** This is about 133× faster than the baseline setting that generates full reasoning for every judgment.
- **Scalable Larger Model:** JudgeLM-33B completes the same validation in about 15 minutes.
- **Practical Impact:** JudgeLM makes large-scale open-ended evaluation much cheaper and faster.

Efficiency Table

Table 3: Efficiency comparison for our JudgeLM and PandaLM on our *val* set. We use a machine with 8 Nvidia-A100 GPUs with 40G memory to evaluate their efficiency.

Methods	model size	GPUs per model	parallel judge?	generate reason?	total time
PandaLM	7B	1	✗	✓	6 hrs 40 mins
Ours.					
JudgeLM	7B	1	✗	✓	6 hrs 40 mins
JudgeLM	7B	1	✗	✗	24 mins
JudgeLM	7B	1	✓	✓	50 mins
JudgeLM	7B	1	✓	✗	3 mins
JudgeLM	13B	1	✓	✗	5 mins
JudgeLM	33B	2	✓	✗	15 mins

Figure 24: JudgeLM greatly reduces the time cost of evaluating response pairs.

Ablation Study: What Actually Helps?

- **Swap Augmentation:** Improves consistency by 5.44 percentage points and reduces position bias.
- **Reference Support:** Gives the judge access to external truth, improving judgments when internal knowledge is insufficient.
- **Reference Drop:** Trains the model to work both with and without reference answers, reducing format overfitting.
- **Key Lesson:** Fine-tuning alone is not enough; bias-aware fine-tuning is necessary.

Reference Support and Reference Drop

Table 6: Ablation study for the reference support and reference drop on our *val* set.

Methods	*ft* w/ ref?	*val* w/ ref?	Agreement ↑ (w/ GPT-4)	Consistency ↑ (w/ swap.)	Bias ↓ toward 1st	Bias ↓ toward 2nd	Delta Bias ↓
matching format.							
baseline	✗	✗	75.87	73.45	19.83	6.72	13.11
baseline	✓	✓	80.15	81.23	11.55	7.22	4.33
mismatched format.							
baseline	✗	✓	73.09	67.75	29.44	2.81	26.63
baseline	✓	✗	75.69	73.40	20.89	5.71	15.18
w/ ref. drop.							
baseline	ref. drop	✗	76.86	77.13	17.30	5.57	11.73
baseline	ref. drop	✓	80.35	81.24	11.48	7.28	4.20

Figure 25: Reference support and reference drop improve reliability across prompt formats.

Conclusion

Summary

- **Main Contribution:** JudgeLM shows that open-source LLMs can be fine-tuned into scalable judges for open-ended evaluation.
- **Why It Matters:** It reduces dependence on closed APIs like GPT-4 and improves reproducibility, privacy, and efficiency.
- **Key Lesson:** Judge models must be trained carefully because they inherit biases such as position, knowledge, and format bias.
- **Extended Capabilities:** JudgeLM also generalizes beyond pairwise comparison to single-answer grading, multiple-answer judging, multimodal evaluation, and multi-turn chat.
- **Final Takeaway:** JudgeLM is not just a judge model; it is a framework for building faster, cheaper, and more reliable LLM evaluators.

Paper 6:
Benchmarking LLM Judges for System Ranking

Table of Contents – Paper 6

The Missing Perspective: System-Level Ranking

The Gap in Existing Judge Benchmarks

- **Current Practice:** LLM judges are commonly evaluated at the *instance level*: can the judge correctly compare individual responses?
- **Real-World Goal:** Researchers often use these judgments to rank entire models, systems, or configurations.
- **The Problem:** A judge that performs well on individual responses does not necessarily produce an accurate overall system ranking.
- **Why This Matters:** Errors may be distributed unevenly across systems, creating unfair rankings even when average instance-level accuracy looks strong.

Instance-Level vs. System-Level Evaluation

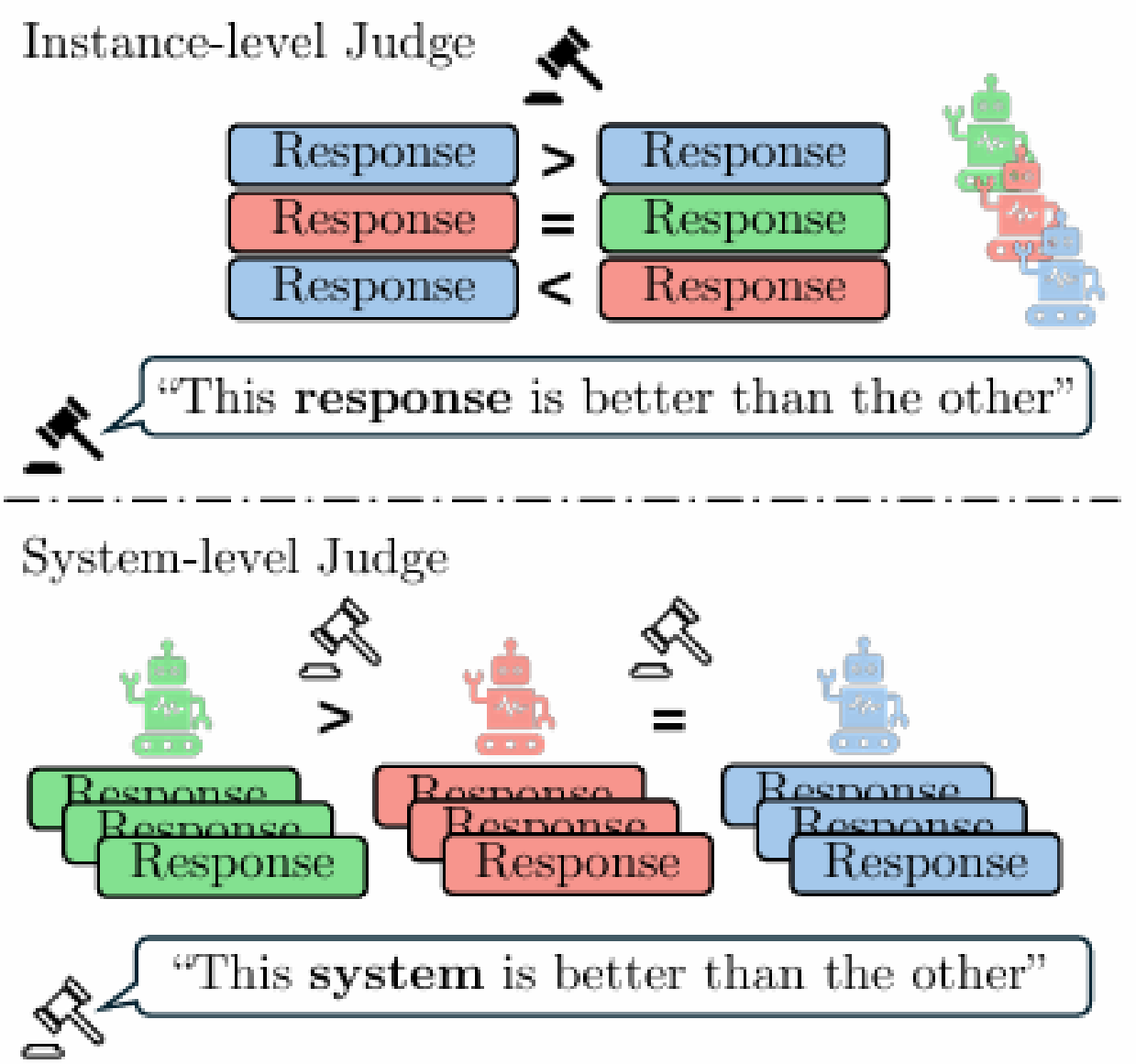


Figure 1: **Instance and system level judges make**

The Main Contribution: JuStRank

- **JuStRank:** A benchmark designed specifically to evaluate judges as *system rankers*.
- **Core Question:** Can an automated judge rank multiple LLM systems similarly to human evaluators?
- **Evaluation Principle:** Aggregate the judge's scores over many outputs, construct a model ranking, and compare it with a human-based gold ranking.
- **Broader Goal:** Help practitioners select the right judge for model comparison and model-selection decisions.

JuStRank Task Formulation & Pipeline

System-Level Judge Pipeline

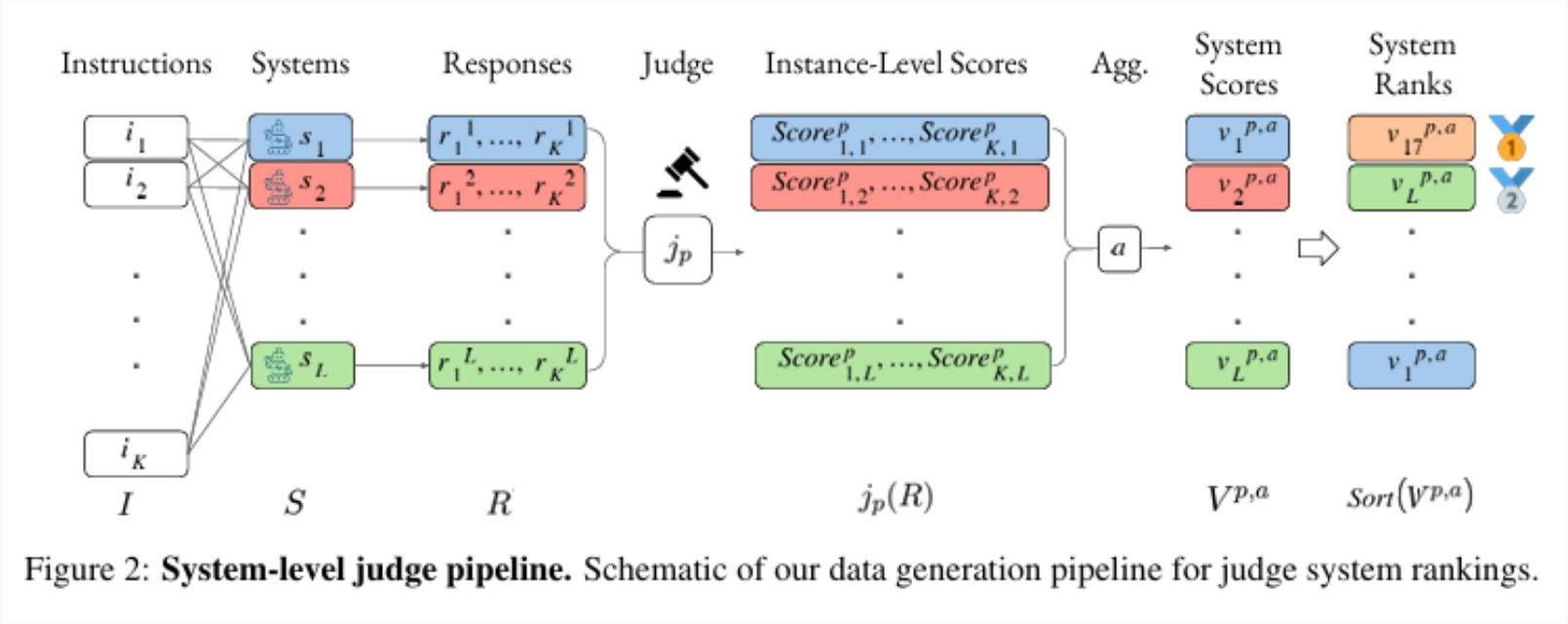


Figure 2: **System-level judge pipeline.** Schematic of our data generation pipeline for judge system rankings.

Figure 27: Pipeline for generating system-level rankings from individual judge scores.

Task Formulation

- Let $S = s_l l = 1^L$ be the set of target systems and $I = i_k k = 1^K$ be the user instructions.
- Each system s_l generates a response r_k^l for instruction i_k.
- A judge j_p assigns a scalar quality score:

$$j_p(i_k, r_k^l) = \text{Score}_{k,l}^p$$

- An aggregation method converts the response-level scores into one score per system:

$$V^{p,a} = a(j_p(R))$$

- The final system ranking is compared with a human-based gold ranking.

Experimental Setup

Experimental Setup

- **System Responses:** Arena Hard v0.1 provides 500 challenging prompts and outputs from 63 systems.
- **Scale:** The benchmark evaluates approximately 32K instruction-response pairs.
- **Judges:** JuStRank studies 48 judge realizations:
  - 8 reward-model judges.
  - 40 generative LLM judges: 10 LLMs evaluated using 4 prompting styles.
- **Total Judgments:** 1.5 million individual scores.
- **Gold Ranking:** Human preference data from the Chatbot Arena English Hard Prompts subset.

Four Ways to Use an LLM as a Judge

- **1. Numeric:** Assign an absolute score between 0 and 100.
- **2. Likert:** Assign a textual rating: Very Bad, Bad, Mediocre, Good, or Very Good.
- **3. TokenProbs:** Answer whether a response is good, then use the probabilities of yes" and no" as a score.
- **4. Anchor:** Compare each system response against an anchor response from GPT-4-0314.

Aggregation Methods:

- Win-Rate, Mean, Median, and Bradley-Terry (BT).

Main Results

Top-Performing System-Level Judges

Judge Model	Realization	Aggregation	Agreement (τ) with Gold Ranking
Qwen2.5-72B-Instruct	Likert	Win-Rate	.83
URM-LLaMa-3.1-8B	Reward	Mean	.82
GPT-4o-2024-11-20	Anchor	Mean	.82
Llama-3-1-405b-instruct-fp8	Numeric	Mean	.81
Mistral-large-instruct-2407	Likert	BT	.81
GPT-4o-mini-2024-07-18	Numeric	Win-Rate	.81
ArmoRM-Llama3-8B-v0.1	Reward	Mean	.80
Llama-3-1-70b-instruct	Numeric	Win-Rate	.80
Skywork-Llama-3.1-8B-v0.2	Reward	Mean	.79
Llama-3.1-8B-Instruct	TokenProbs	Mean	.78

Table 1: **Top 10 judges by ranking performance**. Judges are sorted by the Kendall's Tau correlation between their overall system ranking and the gold ranking from Chatbot Arena (§4.4). For every judge model, only the best-performing realization and aggregation method is shown. For the full results, refer to Appendix Table 2.

Figure 28: Top 10 judges ranked by Kendall's Tau correlation with the human-based gold ranking.

What the Leaderboard Reveals

- **Best Overall Judge:** Qwen2.5-72B-Instruct with Likert scoring and Win-Rate aggregation reaches $\tau = 0.83$.
- **Reward Models Can Compete:** URM-LLaMa-3.1-8B reaches $\tau = 0.82$, comparable to GPT-4o.
- **Size Is Not Everything:** Smaller dedicated reward models can perform similarly to much larger general-purpose LLMs.
- **Practical Lesson:** Selecting a judge requires more than choosing the largest or most famous LLM.

Instance-Level Performance Is Not Enough

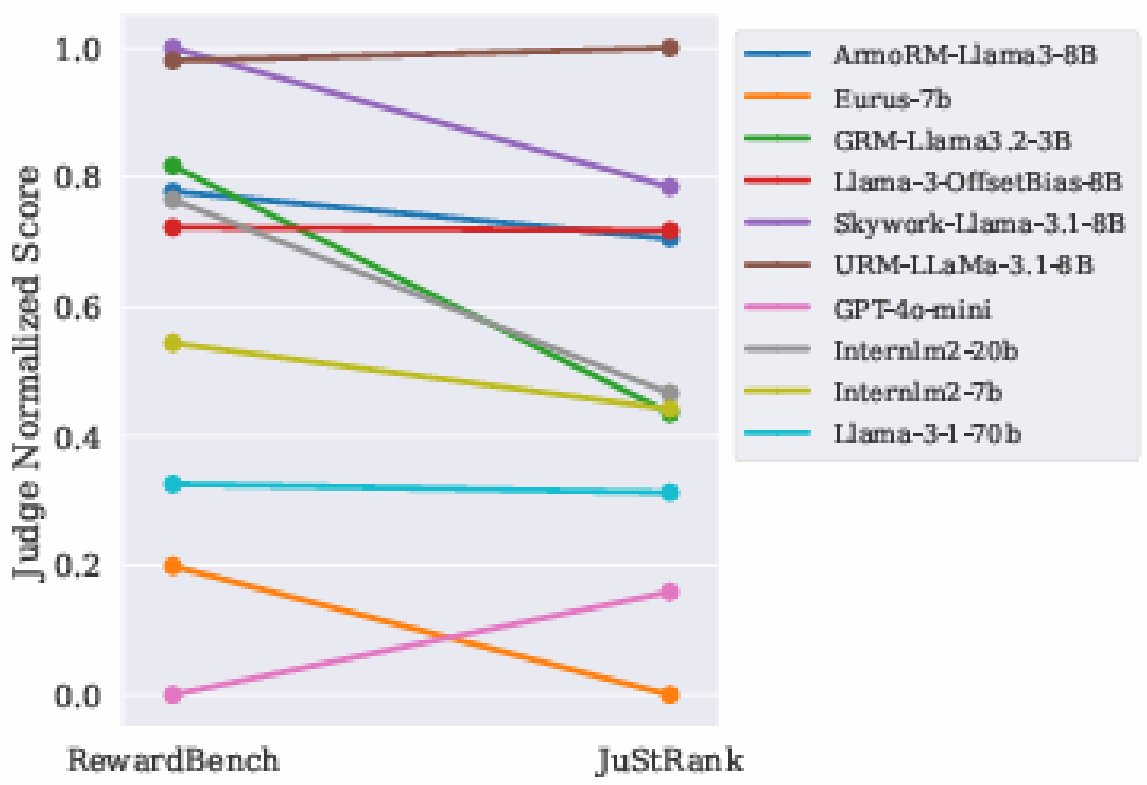


Figure 3: **Comparison to RewardBench**. The plot depicts the relative performance of judges present in both JuStRank and RewardBench (Lambert et al., 2024). For comparison, we perform Min-Max normalization over the judge performance scores (*accuracy* for RewardBench, *Kendall's Tau* for our results). Results shown are for the BT aggregation method: the LLM judges use

The Prompting Style Matters

- **Key Finding:** The judge realization can be almost as important as the identity of the underlying LLM.
- **Strong Choices:** Numeric and Likert scoring generally produce the best system rankings.
- **Unexpected Result:** Comparative Anchor judging is often weaker, except for GPT-4o.
- **Implication:** A strong model can still produce a poor ranking if it is prompted or aggregated badly.

Effect of Judge Realization

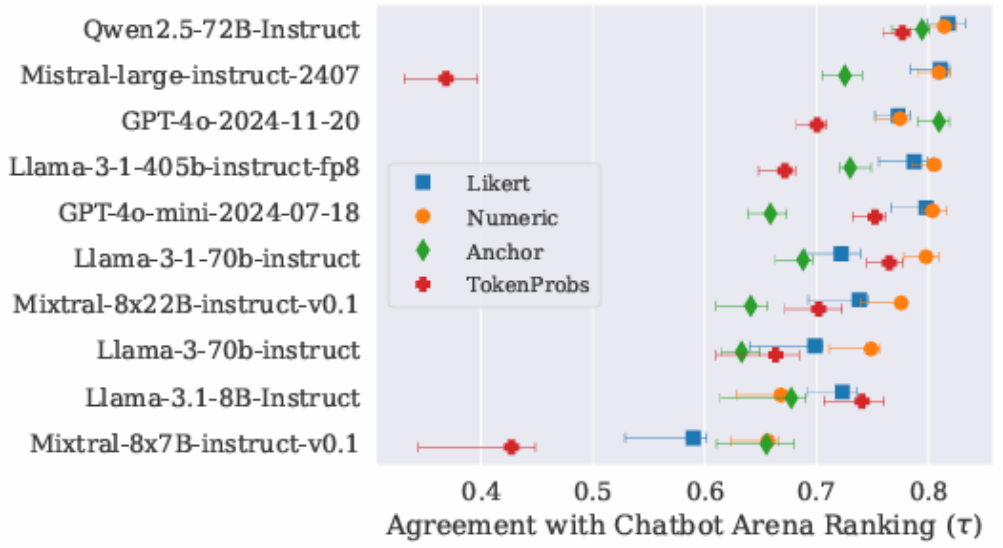


Figure 4: **LLM judge realizations**. Kendall's Tau correlations (±95% bootstrapping CI) between the system rankings produced by various LLM judge realizations (§4.2.2) and the gold system ranking from Chatbot Arena. The plot depicts results for the BT aggregation method; for the full results, refer to App. Table 2.

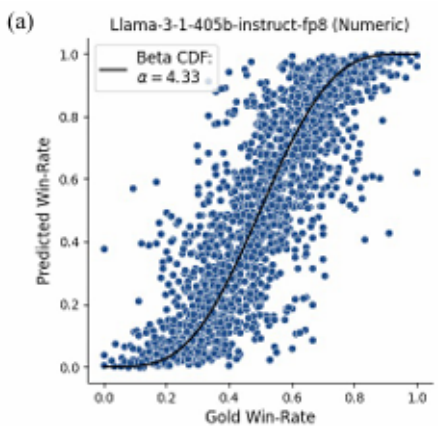
Figure 30: System-ranking quality changes substantially across different LLM judge realizations.

Judge Behavior: Decisiveness & Bias

Emergent Judge Behavior: Decisiveness

- **Decisiveness:** Some judges strongly amplify the performance gap between better and weaker systems.
- **Example:** When humans mildly prefer one system, a decisive judge may produce a much more extreme preference.
- **Potential Benefit:** Decisive judges can separate strong and weak models using fewer examples.
- **Caution:** Greater decisiveness does not always mean greater fairness or calibration.

Visualizing Decisiveness



(b)

LLM Judge Realization

	Likert	Numeric	Anchor	TokenProbs
GPT-4o-2024-11-20	5.5	2.1	3.1	2.2
Mistral-large-instruct-2407	5.5	3.0	2.1	1.2
Llama-3-1-405b-instruct-fp8	5.2	4.3	3.6	1.5
Qwen2.5-72B-Instruct	4.8	4.1	2.9	2.7
GPT-4o-mini-2024-07-18	4.6	2.9	1.4	2.1
Llama-3-1-70b-instruct	3.9	2.7	2.7	1.3
Mixtral-8x22B-instruct-v0.1	2.5	2.1	1.5	1.8
Llama-3-70b-instruct	2.4	1.3	1.8	0.8
Llama-3.1-8B-Instruct	0.9	1.2	0.9	0.6
Mixtral-8x7B-instruct-v0.1	0.8	1.3	1.2	0.7

α

Figure 6: **Beta distribution fit of pairwise win-rates**. (a): *Judge beta fit example*. Each point represents the win-rate between a pair of systems, $WR(s_a, s_b)$; the curve and α value describe a fit to the beta distribution (App. F). Plots for all judges are in App. Fig. 11. (b): *Decisiveness by judge realization*. Cell values denote the decisiveness behaviors of different LLM judge realizations, as described by the α value for their win-rate distribution.

Figure 31: Different judge realizations exhibit different levels of decisiveness.

System-Specific Bias

- **Bias Definition:** A judge is biased toward a system if its predicted win rates are consistently higher or lower than the human-based win rates.
- **Positive Bias:** Some systems receive systematically inflated rankings.
- **Negative Bias:** Other systems are evaluated too harshly.
- **Example:** Many judges show positive bias toward Athene-70B, while GPT-4-0613 receives negative bias.
- **Core Risk:** A biased judge can distort model-selection decisions even when its average accuracy appears acceptable.

System-Specific Bias Heat Map

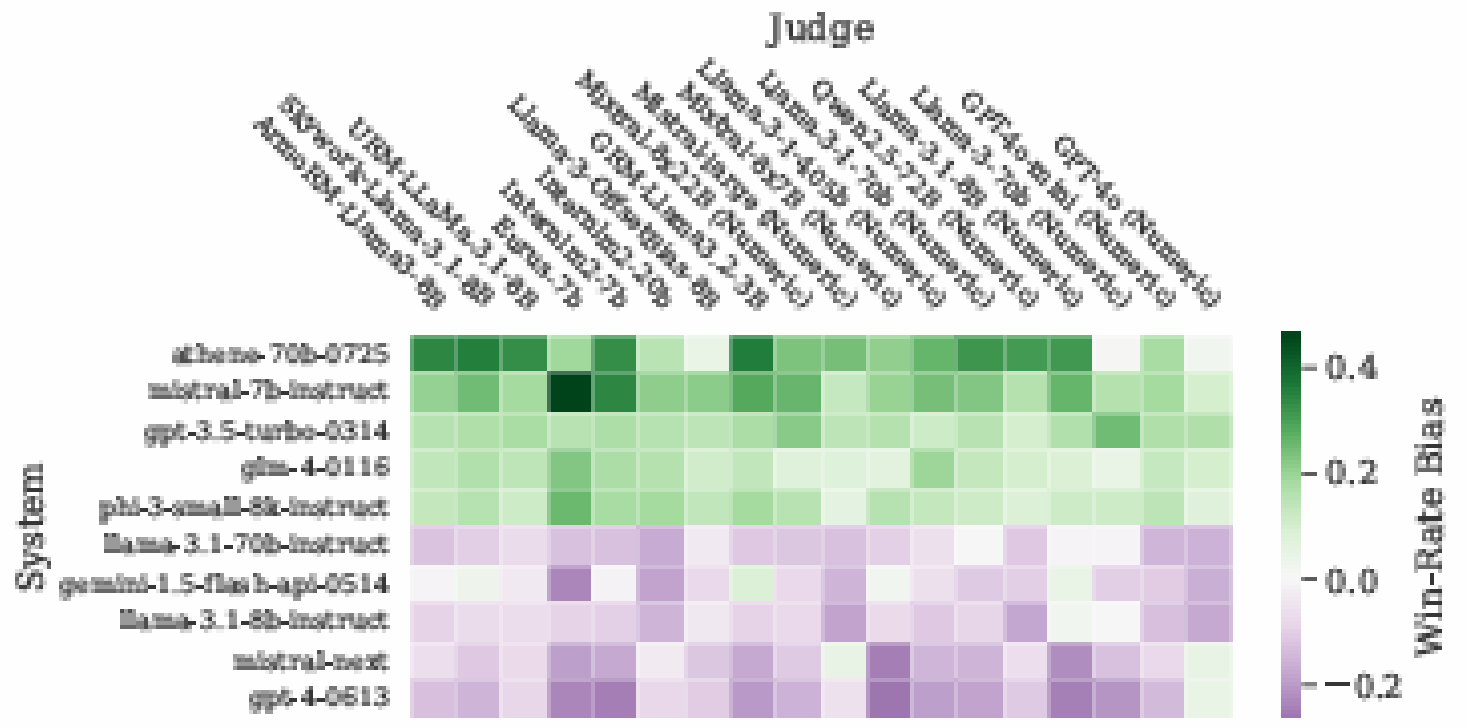


Figure 7: **System-specific judge biases**. The plot depicts win-rate biases of judges towards specific systems, with respect to the ground-truth win-rates from Chatbot Arena (after correction for the beta distribution fit of each judge). This plot portrays select systems with high

Discussion & Conclusion

Discussion and Limitations

- **Prompt Sensitivity:** Judge behavior can change when the realization prompt changes.
- **Human Preference Is Complex:** The gold ranking combines subjective preferences such as helpfulness, safety, style, and coherence.
- **Dataset Alignment:** Judge scores are collected on Arena Hard, while the gold ranking comes from Chatbot Arena's English Hard Prompts subset.
- **Future Work:** Train dedicated system-level judges, explore judge ensembles, and test additional aggregation methods.

Summary

- **The Missing Evaluation Layer:** Instance-level judge quality does not guarantee accurate system rankings.
- **The Contribution:** JuStRank evaluates judges by comparing their model rankings with a human-based gold ranking.
- **Key Findings:**
  1. Reward models can compete with much larger LLM judges.
  2. Prompting style and score aggregation strongly affect ranking quality.
  3. Judges exhibit emergent decisiveness and system-specific bias.
- **Final Takeaway:** The best judge depends on the actual goal. A judge used to select models should be evaluated as a system ranker, not only as an instance-level evaluator.